

Transition from a Plan-Driven Process to Scrum – A Longitudinal Case Study on Software Quality

Jingyue Li
Norwegian University of Science and
Technology
NO-7491, Trondheim, Norway
jingyue@idi.ntnu.no

Nils B. Moe
SINTEF
NO-7465 Trondheim, Norway
nils.b.moe@sintef.no

Tore Dybå
SINTEF
NO-7465 Trondheim, Norway
tore.dyba@sintef.no

ABSTRACT

Although Scrum is an important topic in software engineering and information systems, few longitudinal industrial studies have investigated the effects of Scrum on software quality, in terms of defects and defect density, and the quality assurance process. In this paper we report on a longitudinal study in which we have followed a project over a three-year period. We compared software quality assurance processes and software defects of the project between a 17-month phase with a plan-driven process, followed by a 20-month phase with Scrum. The results of the study did not show a significant reduction of defect densities or changes of defect profiles after Scrum was used. However, the iterative nature of Scrum resulted in constant system and acceptance testing and related defect fixing, which made the development process more efficient in terms of fewer surprises and better control of software quality and release date. In addition, software quality and knowledge sharing got more focus when using Scrum. However, Scrum put more stress and time pressure on the developers, and made them reluctant to perform certain tasks for later maintenance, such as refactoring.

Categories and Subject Descriptors

D.2.9 [Management]: Software quality assurance and Software process models

General Terms

Management and Measurement

Keywords

Empirical Software Engineering, Agile Software Development, Software Quality

1. INTRODUCTION

Traditional plan-driven development approaches emphasize predictability and stability in a project [6]. Agile software development [10] represents a new approach for planning and

managing software projects. Agile development puts less emphasis on up-front plans and strict plan-based control and more emphasis on mechanisms for change management [23]. Agile development relies on people and their creativity rather than on formalized processes [8]. Leadership and collaboration, informal communication and a flexible and participative organizational form, and encouraging cooperative social action are other characteristics of agile software development [24].

Both plan-driven and agile processes have context-dependent advantages and shortcomings [6]. Huo et al. [16] compared the software quality assurance activities in waterfall and agile processes and argued that agile processes would offer better software quality and shorter time-to-market than waterfall processes. Results of some studies [17, 20, 31] supported the argument of [16] and showed that agile processes helped to reduce defect densities. On the contrary, a study by Abrahamsson [1] did not find a reduced defect density from using XP. Although some studies [1, 17, 20, 30] reported improved software development productivity after using agile methods, Harry Sneed warned in a panel [18] that lower development costs in agile projects might lead to higher maintenance costs later. However, few studies have examined whether agile processes actually facilitate software maintenance activities, such as defect fixing, system enhancement and adaption. Although Ilieva et al. [17] investigated the defect fixing efficiency of agile methods, no conclusion was drawn with respect to the impact of agile methodology on defect fixing efficiency.

As more and more software companies are changing from plan-driven processes to agile ones [4], it is important to perform more empirical studies in industry to validate the conclusions of studies [1, 17, 20, 31]. We also need to find out whether agile processes offer better software quality, and most importantly, which practices of an agile process impact software quality and why.

Our objective in this paper is to provide a better understanding of how agile processes affect software quality. To meet this objective, we have conducted a longitudinal case study on how the transition from a plan-driven to an agile process, i.e. Scrum, may affect software quality. More specifically, we sought to answer the following research question:

How does the transition from a plan-driven process to Scrum affect software quality, in terms of defects and defect density, and the quality assurance process?

In Scrum each sprint should produce potential shippable code. We therefore assume that defects should be discovered and be solved

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ESEM'10, September 16-17, 2010, Bolzano-Bozen, Italy.

Copyright 2010 ACM 978-1-4503-0039-01/10/09...\$10.00.

faster than in a plan-driven process. In addition, in a Scrum project, there should be more feedback and cooperation between the developers and between the developers and the product owner. Thus, we assume Scrum should make it easier to implement the “right” functionality from the start. Motivated by these assumptions and previous studies we decided to investigate the research question by examining the following changes when a project was transitioned from a plan-driven process to Scrum:

- The change in the quality assurance process.
- The change in number and type of defects.
- The change in efficiency of fixing defects.

The rest of the paper is organized as follows. Section 2 gives an overview of agile software development and the process of transitioning from plan-driven to agile development. Section 3 presents the study design. Section 4 presents the results, and Section 5 discusses our findings and possible limitations of the study. Section 6 concludes.

2. BACKGROUND

The prevalent software development processes can be divided into plan-driven and agile [6]. Although practitioners and researchers are arguing on the benefits, shortcomings, and application contexts of plan-driven and agile processes [6], more and more software companies are introducing agile software development. A survey by Begel and Nagappan [4], for example, found that, by October 2006, 32% of their investigated teams had already used agile methods.

2.1 Agile Software Development and Scrum

Agile software development comprises a number of practices and methods [12]. Among the most known and adopted agile methods are Extreme Programming (XP) [3] and Scrum [28]. XP focuses primarily on the implementation of software, while Scrum focuses on agile project management [28].

Scrum has not only reinforced the interests in software project management, but also challenged the conventional ideas about such management. Scrum focuses on project management in situations where it is difficult to plan ahead, with mechanisms for “empirical process control”; where feedback loops constitute the core element [28]. Compared with traditional command-and-control oriented management, Scrum represents a radically new approach for planning and managing software projects, because it brings decision-making authority to the level of operational problems and uncertainties.

In Scrum, software is developed by a self-managing team in increments, called “sprints”, starting with planning and ending with a review. A sprint typically lasts from one to four weeks. Features to be implemented in the system are registered in a backlog. Then, the product owner decides which backlog items should be developed in the following sprint. Team members coordinate their work in a daily stand-up meeting. One team member, the Scrum master, is in charge of solving problems that stop the team from working effectively. There are few empirical studies of Scrum in the research literature [10]. Most of the studies are lessons-learned or experience reports, such as [26], with little scientific backing of claims. Case studies of Scrum include examining the combination of XP and Scrum [13], the

overtime amongst developers and customer satisfaction in Scrum [21], experience with Scrum in a cross-organizational development project [9], and understanding the barriers of introducing self-managing teams in Scrum [22].

2.2 Transition from Plan-driven Process to Agile Process

Software practitioners and researchers have investigated the gains and the possible issues of moving from a plan-driven process to an agile one in terms of software defect density and project productivity.

Layman et al. [20] compared defect density (measured by number of defects/lines-of-code) before and after a project introduced XP, and observed that XP led to 65% lower pre-release defect density and 35% lower post-release defect density. Williams et al. [31] found a 40% reduction of defect density after introducing XP in an IBM project. Ilieva et al. [17] observed 13% fewer number of defects after introducing agile methods. However, Abrahamsson [1] did not find a reduced defect density from using XP. Thus, the conclusion of the impact of agile process on software defect density is still inconsistent.

Regarding the relationship between project productivity and Scrum, Sutherland et al. [29] found that the use of Scrum in a company with CMMI level 5 almost doubled the productivity compared with projects using traditional processes. Furthermore, Benefield [5] claimed that Yahoo had experienced 39% productivity gains after the introduction of Scrum in more than 150 teams worldwide. Richard et al. [25] concluded that much of the perceived increase in productivity from using Scrum at 3M came from the focus on delivering functional software.

With respect to the relationship between project productivity and other agile methods, Ilieva et al. [17] reported 42% productivity gain and Layman et al. [20] reported 50% productivity gains by using agile methods. Wellington et al. [30] reported 44% productivity gains by using XP. However, experience from Apple illustrated that improper uses of agile practices could be counter-productive [19]. Rundle and Dewar [27] also indicated that an agile process might not lead to better Return On Investment (ROI) than a plan-driven process. Most previous studies focused on the productivity to produce code by measuring the line-of-code developed. Although Ilieva et al. [17] tried to measure the effort spent on defect fixing, no data and conclusion were presented.

3. RESEARCH DESIGN

To answer our research question and to investigate the impact of agile processes on software quality assurance process and the defects of developed software, we used a longitudinal single-case holistic study [32]. When programmers change from the plan-driven process to Scrum, they usually need time to learn Scrum and to be familiar with it. We chose to perform a longitudinal case study, because we believed that such a study was helpful to avoid the biased comparisons between an established plan-driven process and an immature Scrum process. In addition, we believed that such a study could give us better opportunities to collect qualitative and quantitative data to examine how the change in software development processes impacted software defect density and defect profiles.

3.1 Investigated Company and Project

The context information of the investigated company and project is as follows:

Company: The study was carried out within a large Norwegian company. One large and important part of their operation is their software development with approximately 150 employees in three organizational units. The company produces specialized software for the engineering domain. The company sells mass-market software and also writes customer specific software on a contract-basis. All developers of the project investigated are located at the company's headquarter in Oslo, Norway. The company also conducts software development in its offices in China, Eastern Europe, and UK.

Product and market: The aim of the project was to develop a software system for asset integrity management of off-shore installations. The size of the system built by this project had more than 300 Non-commented Kilo-Lines-of-Code. The project was a fixed price, one customer project. The customer got high confidence in the team, letting them take several of the decisions, such as reprioritizing releases, without being involved too much. This trust made the team strive for delivering code with no post-release defects. So far, the customer reported only one defect after system deployment.

Practice, Tools, Techniques: The software was developed on .NET framework in Visual Studio using C#.

People: The investigated project consisted of one project manager and five developers allocated 100% on the project. Two of these developers were external consultants. Some temporary employees, who contributed in less than 10% of the total effort, also worked on the project to substitute developers occasionally, due to the developers' vacations, sick-leave, or increased workload in certain periods. Most of the developers had more than 10-years software development experience.

Processes: The first three phases of the project used a traditional plan-driven process. This study examined the third phase of the project, which lasted from 1st January 2006 to 31st May 2007. This third phase is called pre-Scrum phase in this paper. In the fourth phase of the project, Scrum was introduced. The project team participated in a two-day introduction course on Scrum. One of the developers worked as a Scrum-master, and the former project manager was the product owner. The product owner was also doing the system and acceptance testing, and handling traditional project management tasks. The Scrum-master of this project got the Scrum-master certification after 6 months of using Scrum.

Introducing Scrum involved iterative development, empirical process control, and continuous system testing, and so on. An overview of the changes from the pre-Scrum phase to the Scrum phase is shown in Table 1.

The first sprint lasted four weeks, but they changed this to 14 days after a couple of sprints. Each sprint ends with a review meeting and starts with a 30 minutes retrospective meeting followed by a 1-2 hours planning meeting. During the planning meeting, tasks are assigned to people on the basis of competence. The team was initially highly specialized, but they also had a focus on letting people try out new areas to reduce their vulnerability and to constantly challenge the developers. The

project manager said: "I need happy people, and therefore I need to think beyond the sprint".

Table 1. Differences between Scrum and pre-Scrum phases

Process/practice	Scrum phase	Pre-Scrum phase
Iterations and Increments	X	
On-Site Customer (*)	X	X
Continuously system and acceptance testing and defect fixing	X	
Detailed planning up front		X
Adaptive planning	X	
Effort estimation up front		X
Empirical process control	X	
Self-managing teams	X	
Retrospective	X	
Daily meetings	X	
Frequent integration	X	
Refactoring	X	

* The project owner represents the customer in our investigated project

In the pre-Scrum phase, the project manager did all system and acceptance testing after all functionalities were implemented. This last period, including system and acceptance testing, and related defect fixing, lasted 6 months. After introducing Scrum, system and acceptance testing were done every 14 days. The system and acceptance testing were still performed manually by the product owner, but defects were now found continuously and not only at the end of a release. When a critical defect was found, it was usually fixed inside the sprint. However, if the defect required too much work and was not critical, it was added to the list of work to be done in the next sprint or in the final sprint.

To illustrate the status and flow of tasks, the project members created a Scrum "wall" (shown in Figure 1). By using the "wall", every task is visible, and categorized into "To Do", "In Progress", "Testing", "Postponed (next sprint)", and "Finished". When the developers complete a feature, it is moved to "Testing". Then the product owner will test it before moving it to "Finished".



Figure 1. The Scrum wall

3.2 Data Collection and Analyses

To answer the research question, we collected qualitative and quantitative data, including interviews and defect reports.

3.2.1 Qualitative data

We participated in and observed the introduction and tailoring of Scrum to the project. We discussed with the project participants to understand how Scrum changed their working processes.

Eight months after Scrum was introduced, the second author interviewed all project members. Another round of interview with all project members was performed 20 months after Scrum was introduced. The product owner was interviewed four times. Each interview lasted from 20 to 50 minutes. The interviews were semi-structured to understand how Scrum was applied and how the introduction of Scrum affected and changed their development and testing process. We focused on understanding testing, work coordination, internal team communication, feedback-sessions, planning and estimation, and decision making.

In addition to the interviews we arranged feedback session after the interview and after the analysis of the defect reports of the project. This was done to make the project members explain and comment on the results of our defect report analyses.

The interviews were transcribed and then imported together with the notes from discussions, feedback sessions and the introduction of Scrum, into a tool (called NVivo) for analyzing qualitative data. In the analysis, we emphasized on how the quality assurance processes had changed, and how different participants in the project interpreted this changes. Material to describe these processes were taken across all sources and synthesized.

3.2.2 Quantitative data

To collect quantitative data, we collected defect reports from the project during the pre-Scrum phase. We also collected data of defects reported from 1st June 2007 to 29th January 2009 during the Scrum phase.

We collected 449 defects reported during the pre-Scrum phase and 895 defects reported during the Scrum phase. For defect analyses, we included only defects that could be reproduced and had been closed.

For defects reported in the pre-Scrum phase, 32 of them in the status “open” or “cancelled” were excluded, because these defects were either not fixed, or were wrongly reported as defects. For defects reported in the Scrum phase, 69 were excluded for the same reasons. For the remaining 417 defects in pre-Scrum phase, we excluded 59 defects, either because they could not be reproduced, or because they were not confirmed as defects and were therefore rejected by developers, or because they were duplications of other defects. Similarly, 130 defects reported in the Scrum phase were excluded. At the end, 358 defects from the pre-Scrum phase and 696 defects from the Scrum phase were regarded as valid and used for our defect report analysis.

The collected defect reports included all defects discovered by the system and acceptance tester. When a defect was detected during such testing, a defect report was written and stored in an in-house built tool. Each defect report contained the following items: an ID, a headline to briefly describe the defect, status (indicating whether the defect has been fixed or not), subsystem location (e.g. one or several components of the system), responsible person to fix the defect, details of the defect, and developers’ comments on the defect (typically also including information about how the defect was fixed), the name of the tester (usually the product

owner), and priority (indicating the urgency of fixing the defect) assigned by the testers. Four priority levels were defined by the company:

- Critical - defects that are very serious and must be fixed quickly in order to meet the deployment requirements.
- High - defects that are serious and must be fixed.
- Medium - defects that are not critical, and there exists a work-around. An evaluation is undertaken to review whether a fix is required or not.
- Low - defects representing a weakness in the code that may cause the inconvenience of using the system or a failure of the system. It is subject to reviews and may result in a correction request.

We classified defects of both pre-Scrum and the Scrum phases into different defect *types*, by using the Orthogonal Defect Classification (ODC) [7]. ODC focuses on tracing each defect back to a specific stage of development. Another popular classification scheme is the one from HP [15] aiming at initiating a process improvement activity to prevent the defect early. Although there are disagreements about the repeatability of ODC [11], we selected ODC, because our research goal was to link defects back to the development practices of a project. To successfully use a defect classification scheme, it is recommended to tailor the scheme based on the company and system [14]. We used the defect *type* and *qualifier* attributes of the ODC and slightly adjusted the values of these two attributes according to our investigated system. The defect *type* and *qualifier* attributes we use and their values are shown in Table 2.

Table 2. Values of the defect type and qualifier attributes

Attributes	Values and definition
Defect type	Functionality: The function of the system is not satisfactory
	Algorithm: Logic and computation problems that can be fixed by (re)implementing an algorithm of a class or component
	Relationship: Problems related to interaction or synchronization between components
	Data: Problems related to structure, content, or declaration of data or files
	GUI: Layout problem of the GUI
	Checking: Problems related to validation of parameters or data in conditional statements.
	Message: Problems related to message presented to the user of the system
Qualifier	Missing: The defect was to due to an omission
	Better: The current implementation is function but is not satisfactory
	Wrong: The defect was to due to a commission

4. RESULTS

In what follows, we will use the data we collected to present our findings on the impact of Scrum on software quality assurance process, software defects, and defect fixing efficiency.

4.1 Software Quality Assurance Process

As mentioned in Section 3.1, the project did not perform any system or acceptances testing during the first 11 months of the

pre-Scrum phase, and consequently did not report or close any defects in this period. In the Scrum phase, there were four releases (the release dates were 29th November 2007, 1st February 2008, 2nd July 2008, and 29th January 2009). System and acceptance testing and related defect fixing were performed in every sprint of a release. Each release ended by a release sprint consisting of final system and acceptance testing and related defect fixing, and to prepare a handover to the customer.

We analyzed the reporting and closing date of defects and calculated the number of defects reported and closed at a certain date, in pre-Scrum and Scrum phases. The results are shown in Figure 2 and Figure 3 respectively. Figure 3 confirms that system and acceptance testing and related defect fixing were done continuously in the Scrum phase. We found the percentages of defects fixed before the release sprint to be 69%, 60%, 54%, and 46% respectively for releases 1 to 4. The reason for leaving defects unsolved until the release sprint was motivated by the need of keeping the pace up. Usually all critical defects were solved while sprinting.

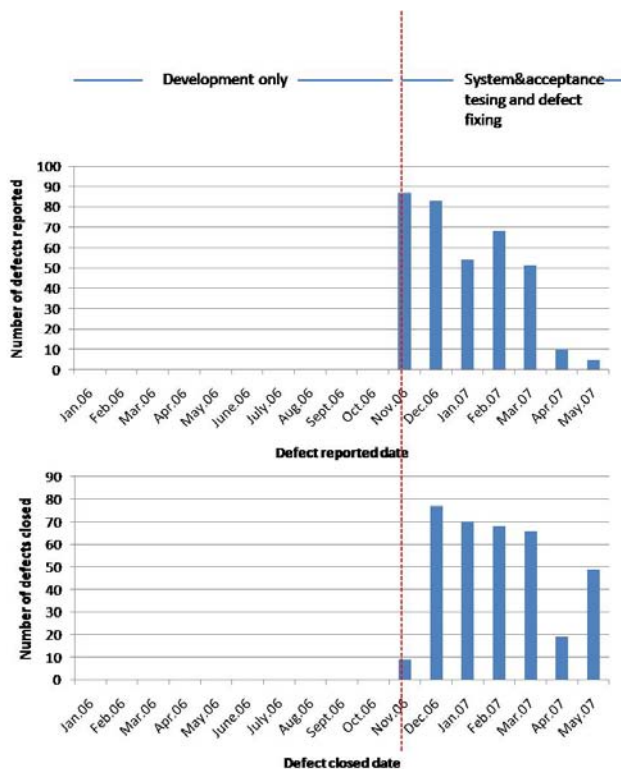


Figure 2. Numbers of defects both reported and closed in the 17 months of the pre-Scrum phase

The new process of finding and solving defects was a big change to the company. The interesting question is then: what is the effect of this change? The answer can be found in the interview data.

During the interviews, the product owner commented that the focus on software quality had increased after introducing Scrum, because of continuous feedback in the daily meetings, short sprints, and sprint retrospectives. She said:

Now [as opposed to the pre-Scrum phase] the project is measured every sprint, and we are focusing on how good this sprint was. Did we deliver what was promised? Was there few or many defects? Did we correct all the defects? There is an increased focus in the whole project about these issues.

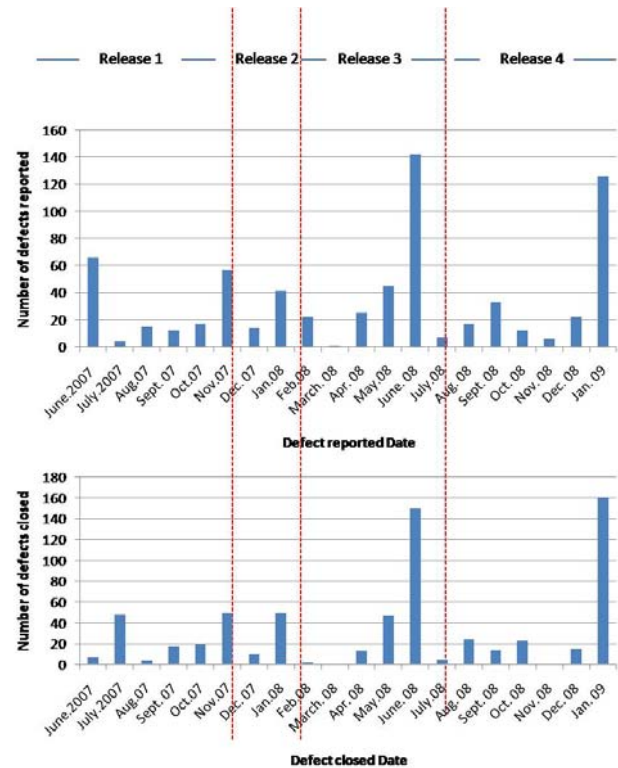


Figure 3. Numbers of defects both reported and closed during the first 20 months after Scrum was introduced

The project owner also felt she got a better overview of the software quality, because critical defects were found and corrected every 14 days. She continued:

There was an enormous list of defects and errors in the last phase of pre-Scrum phase. It was not easy to have a good overview of this list regarding required work to fix them. Also when correcting one defect, another was found. This resulted in new defects being found late in the process.

The team experienced that applying Scrum ensured that defects were dealt with within a reasonable timeframe. As a result, they knew that the amount of remaining defects before deployment was much more manageable compared with the plan-driven process. They also experienced, when using the plan-driven process, they lacked a good understanding of remaining work since, system and acceptance testing was done in the end, and as a result the team had difficulties meeting agreed deadlines. One developer said:

Now, the sum of unexpected "happenings" in the last phase has decreased significantly.

4.2 Defect Profiles

The project owner did not believe that defects introduced by developers were reduced, because of using Scrum. She said:

From my perspective I do not believe that a software development project run with the use of Scrum in itself reduces the defect density much. People are people and errors will be made irrespective of process applied.

The developers, on the other hand, felt that the Scrum process was helpful to reduce the defect density. One developer said:

We felt encouragement to deliver executable and high quality code during each sprint. We were also motivated to do more unit tests on our code before these codes are delivered to the project owner for system or acceptance testing.

Another developer mentioned that Scrum increased communication and overall understanding of the code, and this would reduce the number of defects. She said:

I believe it should be fewer defects after introducing Scrum because we are now communicating more frequently about what we were doing before. Especially the short meetings every morning improve the team communication, and in these meetings we clarify problems and resolve uncertainties regarding the tasks we are solving. If you are stuck you get help.

To examine whether the number of defects was actually reduced due to the Scrum practices, we compared the defect densities of the pre-Scrum phase and the Scrum phase. One option is to use the size of the newly built code divided by the number of defects to measure the defect density. However, we found this measure was problematic to use. Both the pre-Scrum phase and Scrum phase covered a combination of improvements on previous delivered functionalities and development of new functionalities. It was therefore difficult to distinguish newly built code and to measure its size. The team also integrated some third-party components into the system and made it even more difficult to calculate the newly developed code. In addition, the team did some refactoring in a few releases. Code refactoring is the process of changing the source code without modifying its external functional behavior in order to improve some of the nonfunctional attributes of the software. After the refactoring the number of lines-of-code is usually reduced. Therefore we used the number of defects divided by the developers' effort (measured by person-hours) to calculate the defect densities of the pre-Scrum phase and the Scrum phase.

The total effort developers spent on the pre-Scrum phase was 10,768 person-hours. The total effort developers spent in the four releases of the Scrum phase was 16,720 person-hours. Thus, the defect density of the pre-Scrum phase was 33.4 (i.e. 358/10.8) per thousand person-hours, while the defect density of the Scrum phase was 41.7 (i.e. 696/16.7) per thousand person-hours.

Although it looks that the pre-Scrum phase has 25% lower defect densities than the Scrum phase, it does not show the whole story. After we examined the effort developers spent on the last 6 months of the pre-Scrum phase, we found that they used 3,869 person-hours on the fixing the defects found by system and acceptance testing. This was 36 % of all the effort developers spent in the pre-Scrum phase. The team used a lot more hours than expected in this phase. We also believe that the density of defect introduced during defect fixing is lower than the density of defect introduced during new development. Therefore, we probably included too many person-hours, which might have

introduced defects, than actual in the pre-Scrum, and therefore made defect density of the pre-Scrum phase look lower than the Scrum phase. Ideally, we should have used the defect introduced during development to divide effort spent on development only to calculate the defect density. However, the dilemma was that we could not distinguish the effort the developers spent on the development and defect fixing during the Scrum phase, because development and defect fixing were done constantly in a sprint, and the effort were not distinguished in the company's effort tracking system.

In addition to defect densities, we also examined whether the defect type profiles of the pre-Scrum phase were different with those from the Scrum phase. After the defects were type-classified, we compared their distributions. First, we just used the *type* attribute without using the *qualifier* attributes. Results are shown in Figure 4 and illustrate that there is no significant difference of the distributions of different types of defects between pre-Scrum phase and Scrum phase.

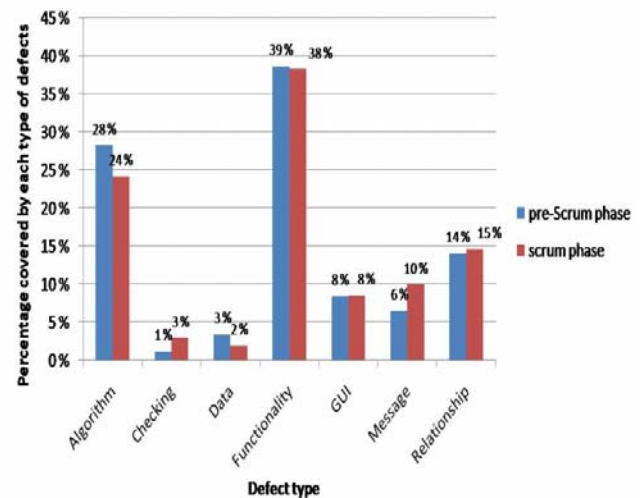


Figure 4. Comparison of distributions of different types of defects between pre-Scrum phase and Scrum phase

Then, we classified defects by using both *type* and *qualifier*. The results are shown in Figure 5. As the results show that the Scrum phase has a slightly higher share of defects with type “wrong functionality” than the pre-Scrum phase, we asked the product owner to explain the phenomenon, because we had expected a lower number in the Scrum phase. She explained that this was because the two consultants were replaced when introducing Scrum. The new hired consultants had limited domain knowledge, and as a result, they found it was difficult to understand the requirements unless specifically stated or explained. This resulted in more defects being reported as “wrong functionality”.

To examine whether there were differences between a mature and immature Scrum, we also compared the defect type-profiles in the four releases of the Scrum phase. The results are presented in Figure 6. Again, we cannot see significant difference of the distributions of different types of defects between releases of the Scrum phase, except a slightly higher “better GUI” share in the third release. The project manager explained that the higher “better GUI” was due to the extra money they got in this release to improve the GUI.

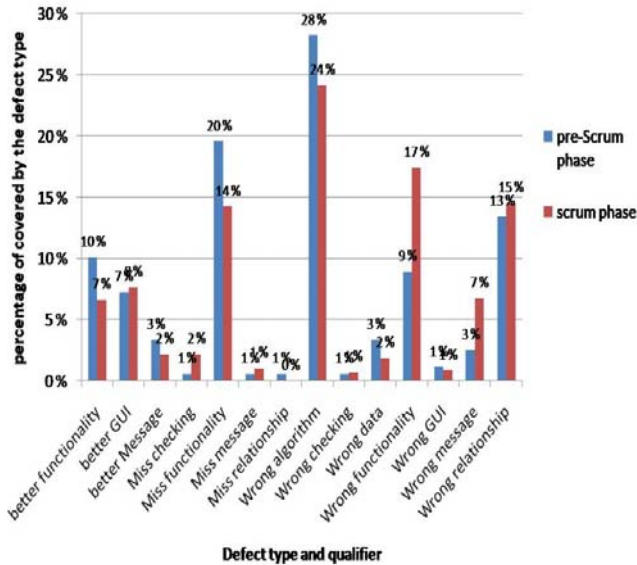


Figure 5. Comparison of distributions of different types of defects with qualifiers between pre-Scrum phase and Scrum phase

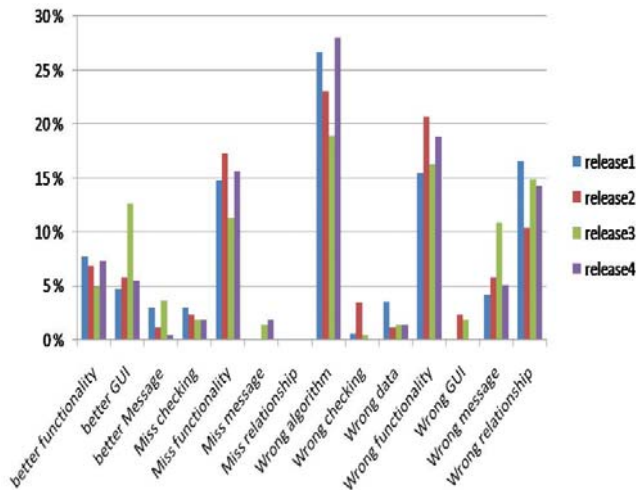


Figure 6. Comparison of distributions of different types of defects with qualifiers between releases of the Scrum phase

In addition, we analyzed the *priority* of defects in the pre-Scrum phase and the Scrum phase. The results show that 47% of defects in the Scrum phase were given the priority value “critical”, while only 10% of defects in the pre-Scrum phase were classified as “critical”. The product owner explained:

The differences of the priority values of the defects do not mean that defects discovered in the Scrum phase are more critical than those discovered in the pre-Scrum phase. In the pre-Scrum phase, the priority value of a defect was assigned when the defect was detected. The priority was never changed afterwards. However, in the Scrum phase, defects were given a priority value when it was discovered. The defects classified as “critical” were usually fixed while sprinting. Defects not classified as “critical” could be postponed to later sprints for fixing. Before we entered the last sprint of a release, I went

through all the defects once more and updated their priorities. The remaining defects were reclassified from “high” or “low” to “critical”. That is why a lot of defects in the Scrum phase were classified as “critical”.

4.3 Defect Fixing Efficiency

To verify whether developers used less time fixing defects in the Scrum phase than in the pre-Scrum phase, we need to know the developers’ effort spent on fixing defects. As mentioned in Section 4.2, in the pre-Scrum phase, 3,869 person-hours were spent by developers to fix defects, which was 36% of the total effort developers spent in the pre-Scrum phase. As explained in Section 4.2, in the Scrum phase the team did not distinguish hours used on development with hours spent on defect fixing, because development and defect fixing were interwove. However, the team members reported in the interviews that they experienced a much better defect fixing performance. The product owner said:

We believe the decreased effort to fix defects is the main benefit of using Scrum. During pre-Scrum phase, the system testing and defect fixing happened at the very late stage of the project. When a defect was discovered, it took a long time to discuss if it was a defect. In addition, it took the developer a lot of time to remember the code they were working on several months ago and to recall which part of the code was possibly related to the defect.

Another said:

We remember very well what we have programmed recently. It is much easier to figure out the reasons for the defect and to correct the defect than before. Another advantage of Scrum is that we [the developers] discuss issues during the daily sprint meeting and get feedbacks or suggestions from others. That also helps in fixing the defects quickly.

Additionally, with Scrum, when someone was missing competence or got stuck solving a task, this would be reported in the daily meetings and constantly during the working day. A team member, who had problems with implementing some code in the system, would immediately ask for help, and then sit together with others to solve the problem. The product owner said:

No one is allowed to waste his working hours by being stuck. You are allowed to use 10 minutes trying to solve the problem, but if you are still stuck you should get up and then ask: do anyone know this? And it is likely that someone else can help you. You lose face if you do not ask for help.

Furthermore, a developer said:

We felt that the team was more protected in the Scrum phase than before. We did not need to jump from one project to another and jump back frequently, as what had happened in pre-Scrum phase. In addition, we felt that the quick feedback and knowledge sharing from the daily sprint meeting have helped us to understand the system better and to learn from mistakes early.

In the pre-Scrum phase, the team lost resources to other projects, as the product owner explained:

It was impossible to protect the team. When other projects ask for resources and they know you are going to deliver in 12

months, it is difficult to deny helping other projects. Now, when using Scrum, we deliver every sprint, and then it is much easier to say no.

However, one developer reported that introducing Scrum increased the pressure because of continuous delivery of running software and defect fixing. He said:

It is like having a pistol against your neck. It's good and bad. You fix things now and not later. But there are also tasks you should have done like code refactoring. I think we do not use enough time on refactoring, because you need to deliver what you promised, and what the team promised.

5. DISCUSSION

The results of our studies give insights to both industrial practitioners and researchers. For industrial Scrum users, our results show that different Scrum practices contributed differently to improve the software quality. For researchers, our studies show that combining quantitative and qualitative methods was helpful to make a credible industrial empirical study.

5.1 Implication for Industry

Our results showed that *iterative development and early testing* of Scrum contributed to manage software defects better. Sutherland et al.'s study [29] showed that early testing of the agile process reduced the amount of remaining code defects in the final test by 42%. Our study showed that around half of the defects, especially the critical ones, were fixed before the last two weeks of the system deployment. This practice made it possible for the project to avoid big repairs at the end of a release and to be able to deliver the product on schedule.

Surveys made by Benefield [5] showed that 68% of respondents believed that Scrum helped to reduce the amount of time wasted. However, the study did not investigate the reasons for such an improvement. Several studies [17, 20, 29-30] have found an increase in the number of lines of code produced when introducing Scrum. In our study, this measure did not make sense because developers copied and pasted code, refactored code, and used third-party components. Therefore, we focused on the productivity of defect fixing. Although we could not precisely quantify the defect fixing efficiency, the results of our interviews still illustrated that *iterative development and early testing* in Scrum helped to improve the efficiency of defect fixing compared to the plan-driven process, because it was easier for developers' to remember code made a few weeks ago than code made several months ago. *Knowledge sharing, retrospective and daily meetings* also helped to improve defect fixing efficiency, since it was easy to report problems and to get help when solving a problem. These results are consistent with findings from Richard et al. [25].

The *empirical process control* of Scrum made it easier to keep a high focus on software quality than the plan-driven process. The late testing in the plan-driven process resulted in late discovery of defects, corresponding budget and schedule overrun, and little control over software quality before very late in the process.

Although other studies [17, 20, 31] concluded that agile methods helped to reduce defect densities, our results did not support this observation. However, as we could not measure the new

developed lines-of-code because of problems related to copy and paste, third party components, and refactoring, our results were not directly comparable with these studies. We found a lower defect density in the pre-Scrum phase than in the Scrum phase. However this was explained by how the defect density was measured. The amount of extra hours needed to do the final defect fixing in the pre-Scrum phase resulted in lower density as measured by the number of defects per hour.

Our study investigated a commercial company for more than three years. During such a long time, it was unavoidable to have sick leave of personnel and changes of developers. These changes might have affected the results, like the previously stated increase in "wrong functionality" defects (see Fig. 5). Baskerville et al. [2] argued that an agile process was probably worse to deal with team members changes than a plan-driven development process. That can partially explain why the project owner felt that the overall defect density of the project was not changed after using Scrum.

Khramov [19] observed that there was no positive correlation between code quality and software project success. Khramov [19] stated that how well the project could deal with volatile of requirements, how efficient the defect fixing was, and some other factors, would in practice decide the project success more than the code quality itself. Therefore, we would argue that the Scrum process helped to improve the chance of project success due to the early defect discovering and improved defect fixing efficiency compared with the plan-driven process, even if the defect densities were not reduced much.

5.2 Implication for Research

Although several previous studies have concluded that agile process improve software quality and productivity, most studies, such as [5, 17], relied solely on quantitative data. Therefore, they could not figure out which Scrum practices impact software quality and productivity to guide industrial practitioners to adapt such practices effectively. We found it was necessary to supplement quantitative data with qualitative studies to better understand which practices of the Scrum process impacted software defects and defect fixing efficiency in what way and why. Relying on only quantitative data in our study would not get the right conclusions. We also found it is problematic to use either number of lines-of-code or effort to measure the defect density.

Additionally, we found that quantitative data are valuable to validate observations and claims from qualitative studies. For example, our interviews with developers showed that they were confident that Scrum reduced defect densities, while the product owner had a different opinion. Our quantitative analysis of the defect type profiles supported the product owners' observations and showed no differences with respect to software defect profiles between the pre-Scrum and Scrum phases. By showing the results to the project members and by asking them, we figured out that several factors, such as personnel changes and insufficient domain knowledge, also influenced the overall software quality of a large and long-term system. Without quantitative analysis, it would be difficult for us to decide whose claim on software quality that best reflected the situation.

5.3 Possible Limitations of the Study

The main limitations of our study were caused by the nature of the industrial data and by our data analyses. Our study was a longitudinal single-case holistic study. Although we could decide which data we wanted to collect and analyze, we could not force developers to collect the data for our own research purpose, because it might influence the productivity of the project negatively. Thus, we could not ask developers to divide the defect fixing effort with development effort in the Scrum phase for our research purpose, because this functionality did not exist in the effort-tracking system of the company. Although the developers of the project felt that the defect fixing efficiency had been greatly improved after using Scrum, we did not have solid data to support such a claim. We also excluded some registered defects with “open” status that might slightly have changed the results of our research question.

One possible threat to the trustworthiness of our data analysis is that some defects discovered during the pre-Scrum phase may have been introduced by development in prior phases. Some defects introduced during the Scrum phase may similarly be discovered after this phase. As the customer of the system reported only one defect after the system was deployed by the first three phases of the project, we generally believe that the system and acceptance testing were solid and few defects had escaped from one phase to the next phase.

Another possible threat to the trustworthy of our data analysis is that the functionalities and complexity of the system may have changed over time. According to the project manager, the complexities of the system implemented in the pre-Scrum and the Scrum phase were not significantly different.

6. CONCLUSION

This aim of this study was to provide a better understanding of how agile processes affect software quality process, software defects, and defect fixing efficiency. The results show that Scrum may not lead to a lower defect density than a plan-driven process. However, the Scrum process helped to increase the success opportunity of the project, because:

- The short sprints mitigate the risk of not resolving issues promptly. Defects are discovered and corrected much earlier by using Scrum than by using the plan-driven process, which gives the team better control over the project and improved defect fixing efficiency.
- Daily Scrum meetings facilitate the knowledge sharing between project members. The quick feedbacks help developers to understand the system better and to learn from mistakes early, which is also helpful to improve the defect fixing efficiency.

However, Scrum makes developers feel more stress to deliver functionalities on time and budget than the plan-driven process. This may make developers reluctant to perform certain tasks to ease later maintenance. So, future studies should investigate how to balance the high pressure of delivering functionalities with the need to perform supplementary tasks.

7. ACKNOWLEDGMENTS

This study was supported by the Research Council of Norway through the project EVISOFT (174390/I40). We would like to thank all project members that participated in this study.

8. REFERENCES

- [1] Abrahamsson, P. Extreme Programming: First Results from a Controlled Case Study. In Proceedings of the 29th Conference on EUROMICRO (Belek-Antalya, Turkey, Sept., 2003). IEEE Computer Society, 259. doi=<http://doi.ieeecomputersociety.org/10.1109/ISESE.2004.1334895>
- [2] Baskerville, R., Ramesh, B., Levine, L., Pries-Heje, J. and Slaughter, S. Is Internet-Speed Software Development Different? IEEE Software, 20, 6 (Nov. 2003), 70-77. doi=<http://dx.doi.org/10.1109/MS.2003.1241369>
- [3] Beck, K. and Andres, C. Extreme Programming Explained: Embrace Change (2nd Edition). Addison-Wesley, 2004.
- [4] Begel, A. and Nagappan, N. Usage and Perceptions of Agile Software Development in an Industrial Context: An Exploratory Study. In Proceedings of the First International Symposium on Empirical Software Engineering and Measurement (Madrid, Spain, Sept., 2007). IEEE Computer Society, 255-264. doi=<http://dx.doi.org/10.1109/ESEM.2007.85>
- [5] Benefield, G. Rolling Out Agile in a Large Enterprise. In Proceedings of the 41st Annual Hawaii International Conference on System Sciences (Waikoloa, Big Island, Hawaii, Jan., 2008). IEEE Computer Society, 461. doi=<http://dx.doi.org/10.1109/HICSS.2008.382>
- [6] Boehm, B. and Turner, R. Using Risk to Balance Agile and Plan-Driven Methods. IEEE Computer, 36, 6 (June 2003), 57-66. doi=<http://dx.doi.org/10.1109/MC.2003.1204376>
- [7] Chillarege, R., Bhandari, I. S., Chaar, J. K., Halliday, M. J., Moebus, D. S., Ray, B. K. and Wong, M.-Y. Orthogonal Defect Classification-A Concept for In-Process Measurements. IEEE Transactions on Software Engineering, 18, 11 (Nov. 1992), 943-956. doi=<http://dx.doi.org/10.1109/32.177364>
- [8] Cockburn, A. and Highsmith, J. Agile Software Development: The People Factor. IEEE Computer, 34, 11 (Nov. 2001), 131-133. doi=<http://dx.doi.org/10.1109/2.963450>
- [9] Dingsøyr, T., Hanssen, G. K., Dybå, T., Anker, G. and Nygaard, J. O. Developing Software with Scrum in a Small Cross-Organizational Project. In Proceedings of the 13th European Conference on Software Process Improvement (Joensuu, Finland, Oct, 2006). Springer Verlag, 5-15. doi=10.1007/11908562_2
- [10] Dybå, T. and Dingsøyr, T. Empirical Studies of Agile Software Development: A Systematic Review. Information and Software Technology, 50, 9-10 (Aug. 2008), 833-859. doi=<http://dx.doi.org/10.1016/j.infsof.2008.01.006>
- [11] Emam, K. E. and Wiczorek, I. The Repeatability of Code Defect Classifications. In Proceedings of the 9th International Symposium on Software Reliability

- Engineering (Paderborn, Germany, Nov., 1998). IEEE Computer Society.
doi=<http://doi.ieeecomputersociety.org/10.1109/ISSRE.1998.730897>
- [12] Erickson, J., Lyytinen, K. and Siau, K. Agile Modeling, Agile Software Development, and Extreme Programming: The State of Research. *Journal of Database Management*, 16, 4 (Dec. 2005), 88-99.
 - [13] Fitzgerald, B., Hartnett, G. and Conboy, K. Customising Agile Methods to Software Practices at Intel Shannon. *European Journal of Information System*, 15, 2 (Apr. 2006), 200-213.
doi=<http://dx.doi.org/10.1057/palgrave.ejis.3000605>
 - [14] Freimut, B., Denger, C. and Ketterer, M. An Industrial Case Study of Implementing and Validating Defect Classification for Process Improvement and Quality Management. In *Proceedings of the 11th IEEE International Software Metrics Symposium (Como, Italy, Sept., 2005)*. IEEE Computer Society, 19.
doi=<http://dx.doi.org/10.1109/METRICS.2005.10>
 - [15] Grady, R. B. *Practical Software Metrics For Project Management And Process Improvement*. Prentice Hall, 1992.
 - [16] Huo, M., Verner, J., Zhu, L. and Babar, M. A. Software Quality and Agile Methods. In *Proceedings of the 28th Annual International Computer Software and Applications Conference - Volume 01 (Hong Kong, China, Sept., 2004)*. IEEE Computer Society, 520-525.
doi=<http://doi.ieeecomputersociety.org/10.1109/CMPSAC.2004.1342889>
 - [17] Ilieva, S., Ivanov, P. and Stefanova, E. Analyses of an Agile Methodology Implementation. In *Proceedings of the 30th EUROMICRO Conference (Rennes, France, Aug., 2004)*. IEEE Computer Society, 326-333.
doi=<http://dx.doi.org/10.1109/EUROMICRO.2004.14>
 - [18] Kajko-Mattsson, M., Lewis, G. A., Siracusa, D., Nelson, T., Chapin, N., Heydt, M., Nocks, J. and Snee, H. Long-term Life Cycle Impact of Agile Methodologies. In *Proceedings of the 22nd IEEE International Conference on Software Maintenance (Philadelphia, Pennsylvania, USA, Sept., 2006)*. IEEE Computer Society, 422-425.
doi=<http://dx.doi.org/10.1109/ICSM.2006.34>
 - [19] Khranov, Y. The Cost of Code Quality. In *Proceedings of the AGILE 2006 (Minneapolis, Minnesota, July, 2006)*. IEEE Computer Society, 119-125.
doi=<http://dx.doi.org/10.1109/AGILE.2006.52>
 - [20] Layman, L., Williams, L. and Cunningham, L. Exploring Extreme Programming in Context: An Industrial Case Study. In *Proceedings of the Agile Development Conference (Salt Lake City, Utah, USA, June, 2004)*. IEEE Computer Society, 32-41.
doi=<http://doi.ieeecomputersociety.org/10.1109/ADEV.2004.15>
 - [21] Mann, C. and Maurer, F. A Case Study on the Impact of Scrum on Overtime and Customer Satisfaction. In *Proceedings of the Proc. of the Agile Development Conference (Denver, USA, July 2005, 2005)*. IEEE Computer Society, 70-79.
doi=<http://dx.doi.org/10.1109/ADC.2005.1>
 - [22] Moe, N. B., Dingsoyr, T. and Dybå, T. Overcoming Barriers to Self-Management in Software Teams. *IEEE Software*, 26, 6 (Nov. 2009), 20-26.
doi=<http://dx.doi.org/10.1109/MS.2009.182>
 - [23] Nerur, S. and Balijepally, V. Theoretical Reflections on Agile Development Methodologies. *Communication of the ACM*, 50, 3 (March 2007), 79-83.
doi=<http://doi.acm.org/10.1145/1226736.1226739>
 - [24] Nerur, S., Mahapatra, R. and Mangalaraj, G. Challenges of Migrating to Agile Methodologies. *Communication of the ACM*, 48, 5 (May 2005), 72-78.
doi=<http://doi.acm.org/10.1145/1060710.1060712>
 - [25] Richard, M., Kelly, R., James, G. and Brian, H. Scrum at a Fortune 500 Manufacturing Company. In *Proceedings of the AGILE 2007 (Washington, DC, Aug., 2007)*. IEEE Computer Society, 175-180.
doi=<http://dx.doi.org/10.1109/AGILE.2007.53>
 - [26] Rising, L. and Janoff, N. S. The Scrum Software Development Process for Small Teams. *IEEE Software*, 17, 4 (July 2000), 26-32.
doi=<http://dx.doi.org/10.1109/52.854065>
 - [27] Rundle, P. J. and Dewar, R. G. Using Return on Investment to Compare Agile and Plan-driven Practices in Undergraduate Group Projects. In *Proceedings of the 28th international conference on Software engineering (Shanghai, China, May, 2006)*. ACM, 649-654.
doi=<http://doi.acm.org/10.1145/1134285.1134383>
 - [28] Schwaber, K. and Beedle, M. *Agile Software Development with SCRUM*. Prentice Hall, 2001.
 - [29] Sutherland, J., Jakobsen, C. R. and Johnson, K. Scrum and CMMI Level 5: The Magic Potion for Code Warriors. In *Proceedings of the AGILE 2007 (Washington, DC, Aug., 2007)*. IEEE Computer Society, 272-278.
doi=<http://dx.doi.org/10.1109/AGILE.2007.52>
 - [30] Wellington, C. A., Briggs, T. and Girard, C. D. Comparison of Student Experiences with Plan-driven and Agile Methodologies. In *Proceedings of the 35th Annual Conference on Frontiers in Education (Indianapolis, Indiana, USA, Oct., 2005)*, T3G-18.
 - [31] Williams, L., Krebs, W., Layman, L., Antón, A. I. and Abrahamsson, P. Toward a Framework for Evaluating Extreme Programming. In *Proceedings of the 8th International Conference on Empirical Assessment in Software Engineering (Edinburgh, Scotland, UK, May, 2004)*, 11-20.
 - [32] Yin, R. K. *Case Study Research: Design and Methods (4th Edition)*. Sage Publications, 2008.